ARTIFICIAL INTELLIGENCE ESSENTIALS

PROJECT REPORT

(Project Semester January-June 2026)

CINEMATCH — MOOD-BASED MOVIES & WEB SERIES RECOMMENDATION SYSTEM

Submitted by

Vandamasu Naga Venkata Ramdev **Roll no. 62** **Registration Number :12414400**

Musini Sri Satya Naga Venkatesh **Roll no. 64** **Registration Number :12408520**

Program and Section P132: 424IM

Course Code INT428

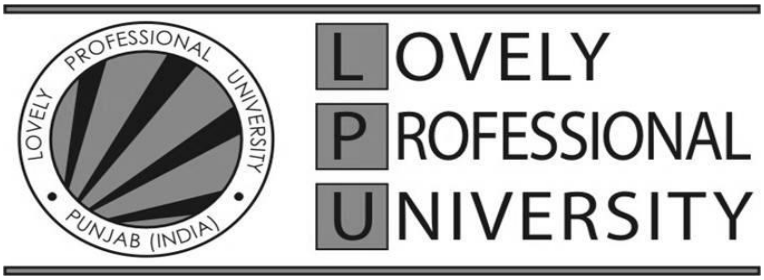
Under the Guidance of

Tarun Singh (U. Id: 34442), Assistant Professor

Discipline of CSE/IT

School of Computer Science and Engineering

Lovely Professional University, Phagwara



DECLARATION

I, Vandamasu Naga Venkata Ramdev, student of B.Tech. Computer Science and Engineering under CSE/IT Discipline at Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.

This project titled " CineMatch — Mood-Based Movies & Web Series Recommendation System" has been completed under the guidance of Tarun Singh, Assistant Professor, Discipline of CSE/IT, as a requirement of INT428 (AI Essentials) for the Project Semester January–June 2026.

No part of the work has ever been submitted for any other degree at any University.

Date: 04-05-2026

Signature: _____
Vandamasu Naga VenkataRamdev
Registration No. 12414400

Signature: _____
Musini Sri Satya Naga Venkatesh
Registration No. 12408520

CERTIFICATE

This is to certify that Vandamasu Naga Venkata Ramdev bearing Registration No. 12414400 has completed INT428 project titled "CineMatch — Mood-Based Movies & Web Series Recommendation System" under my guidance and supervision during the Project Semester January–June 2026 at Lovely Professional University, Phagwara.

To the best of my knowledge, the present work is the result of my original development, effort and study. No part of the work has ever been submitted for any other degree at any University.

The project report is fit for submission and partial fulfillment of the conditions for the award of B.Tech degree in Computer Science and Engineering from Lovely Professional University, Phagwara.

Signature and Name of the Supervisor: Tarun Singh
Designation of the Supervisor
School of Computer Science and Engineering
Lovely Professional University
Phagwara, Punjab.

Date: 04-05-2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty mentor, Tarun Singh (U.Id: 34442), Assistant Professor, Discipline of CSE/IT, Lovely Professional University, for his invaluable guidance, constant support, and encouragement throughout the development of this project.

I would like to specially acknowledge the contribution of The Movie Database (TMDB) for providing free and comprehensive API access that made the data pipeline of CineMatch possible.

I would also like to thank Lovely Professional University for providing the necessary infrastructure, resources, and academic platform to carry out this project successfully.

The development of CineMatch involved solving complex technical challenges — including TF-IDF index alignment with scipy sparse matrices, event bubbling in single-page applications, atomic cache writes to prevent server restarts, and building separate personalisation profiles for movies and web series. Each challenge deepened my understanding of real-world AI/ML system design.

Finally, I extend my gratitude to everyone who directly or indirectly supported and contributed to the successful completion of this project.

Date: 04-05-2026

Vandamasu Naga Venkata Ramdev
Registration No. 12414400
Roll No. 62

Musini Sri Satya Naga Venkatesh
Registration No. 12408520
Roll No. 64

TABLE OF CONTENTS

ABSTRACT

CineMatch is a mood-powered movie and television show recommendation system that uses Natural Language Processing (NLP) and interaction-based personalization to deliver context-aware content suggestions. The system features a Fast-API-based REST backend integrated with a SQLite database, initially populated with over 1,000 titles from The Movie Database (TMDB) API and updated periodically.

At its core, CineMatch applies TF-IDF vectorization with cosine similarity to match a user's mood input against combined title, genre, and description text. A multi-signal scoring mechanism combines mood relevance, user preferences derived from Like/Pass interactions, and content popularity, gradually transitioning from a cold-start model to a personalized recommendation profile.

The recommendation engine also incorporates genre diversity and dislike penalization to improve recommendation quality. The frontend is a single-page application featuring 22 mood themes with a glass-morphism design system, along with language filtering and a "Surprise Me" mode. The system is deployed on Render as a live web application, accessible without installation.

1. INTRODUCTION

1.1 BACKGROUND

The modern entertainment landscape presents users with an overwhelming number of movies and TV shows across platforms like Netflix, Prime Video, DisneyPlusHotstar, and others. Users spend significant time browsing without finding content that genuinely matches their emotional state or personal taste. Traditional recommendation systems rely on collaborative filtering or simple genre tags, failing to capture the nuanced emotional context a user brings when choosing what to watch.

The challenge of personalized content discovery has 2 dimensions:

1)matching the user's current mood or emotional state to appropriate content.

2) Learning from user interactions over time to improve future recommendations. No existing free solution combines both dimensions in a unified, accessible application.

1.2 PROJECT OVERVIEW

CineMatch is a AI/ML web application that solves the content discovery problem by accepting a mood input from the user and delivering personalized Movies and Web Series with Anime/Cartoons recommendations. The system uses TF-IDF vectorization with Cosine-Similarity to match mood descriptions against Movies/Web-Series descriptions and genres. Over time, the system learns the user's taste profile through explicit feedback (Like/Pass) and adjusts recommendations accordingly.

The backend is built with Fast-API (Python), data is sourced dynamically from The Movie Database (TMDB) API, and the frontend is a glass-morphism PWA built with Vanilla HTML, CSS, and JavaScript. The system maintains separate user profiles for movies and Web Series, and automatically refreshes the dataset every Saturday-Midnight.

1.3 OBJECTIVES

1.3.1 To build a mood-based recommendation system that matches user mood descriptions against movie and TV show descriptions using TF-IDF and cosine similarity.

1.3.2 To implement a real-time personalization engine that learns user taste profiles separately for movies and TV shows through Like/Pass interactions.

1.3.3 To develop a TMDB-powered auto-fetching data pipeline that populates and refreshes the dataset without manual intervention.

1.3.4 To create a Glass-Morphism PWA frontend with 22 mood themes that shift's the entire UI colour palette based on the selected mood.

1.3.5 To support language-based filtering with auto-detection of all languages present in the dataset.

2. PROFILE OF THE PROBLEM

2.1 PROBLEM STATEMENT

Design and develop an AI/ML-powered web application that accepts a mood description from the user and uses TF-IDF vectorization with cosine similarity to recommend movies and TV shows whose descriptions match the emotional tone of the input. The system must learn user preferences over time through explicit feedback and maintain separate personalization profiles for movies and web series.

2.2 SCOPE OF THE STUDY

Aspect	Scope
Content Types	Movies and Web Series (TV Shows)
Data Source	TMDB API — auto-fetched on first run, refreshed weekly
Mood Input	22 predefined mood buttons + free-text input
Language Filter	All languages auto-detected from datasets (Optional)
Personalization	Per-User, separate profiles for movies and TV
Platform	PWA — installable on desktop and mobile
Cold Start	Popularity-based until 5 interactions; then personalized
Dataset Size	~500 movies + ~500 TV shows initially; grows weekly

2.3 RATIONALE

1. Mood Gap: No existing platform matches a user's current emotional state to movie descriptions using NLP-based similarity.

2. Personalization Over Time: Collaborative filtering requires large user bases. CineMatch uses individual feedback to build per-user profiles without needing other users' data.

3. Free and Fresh Data: TMDB provides a comprehensive, free API with hundreds of thousands of titles including posters, ratings, and descriptions.

4. Unified Experience: CineMatch combines mood matching, personalization, language filtering, and PWA installation in a single application.

3. EXISTING SYSTEM

3.1 INTRODUCTION

Several platforms exist for Movie/TV recommendations, but each has significant limitations when it comes to mood-based discovery and real-time personalization without a large user base.

3.2 EXISTING SOFTWARE

Platform	Recommendation Method	Limitation
Netflix	Collaborative filtering + watch history	No mood-based input; requires large history
IMDb	Genre/rating filters	No personalization; no mood matching
Letter-Boxed	Social + manual lists	No AI/ML; no mood-based suggestions
JustWatch	Platform aggregator [API]	No recommendations; only search based
Google Discover	Browse history	Not movie-specific; no mood input

3.3 WHAT'S NEW IN THE PROPOSED SYSTEM

• Mood-to-Description Matching: Uses TF-IDF + Cosine-Similarity to match mood text against movie descriptions — not just genre tags.

• Description Keyword Enrichment: Mood vector is enriched with emotional keywords (e.g., 'sad' → grief, loss, heartbreak) for deeper semantic matching.

• Per-User Taste Profiles: Separate genre affinity scores for movies and TV, updated with recency weighting after every Like/Pass.

• TMDB Auto-Fetch Pipeline: No manual dataset maintenance; data fetches and refreshes automatically.

• 22 Mood Themes: The entire UI color palette shifts when a mood is selected — happy turns green, dark turns monochrome, romantic turns rose, etc.

• PWA: Installable on mobile and desktop; works offline for cached pages.

• Language Filter: Auto-detects all languages in the dataset and allows multi-language filtering.

4. PROBLEM ANALYSIS

4.1 PRODUCT DEFINITION

CineMatch is an AI/ML-powered web application that:

• Accepts a mood description (predefined or free-text) from the user.

• Vectorises the mood using TF-IDF with emotional feature-engineering.

• Computes cosine-similarity against movie/TV descriptions from a SQLite database populated via TMDB-API.

• Returns top 10 recommendations with poster images, genre pills, ratings, and a personalization tag.

• Records Like/Pass interactions to build per-user, per-content-type genre preference profiles.

• Shifts from popularity-based to personalized recommendations after 5 interactions.

• Refreshes the dataset every Saturday midnight via AP-Scheduler.

Target Users: Any user on desktop or mobile who wants mood-relevant Movies / Web Series recommendations that improve with use.

4.2 FEASIBILITY ANALYSIS

4.2.1 Technical Feasibility

TF-IDF with scikit-learn is a proven, lightweight NLP technique suitable for this use case. Fast-API provides high-performance async endpoints. SQLite requires no server setup. The TMDB API is free and well-documented. All tools are open-source.

4.2.2 Economic Feasibility

Total project cost is effectively zero. TMDB API is free. Fast-API, SQLite, scikit-learn, and all Python libraries are open-source. Render.com provides free hosting. Development required no software licensing costs.

5. SOFTWARE REQUIREMENT ANALYSIS

5.1 FUNCTIONAL REQUIREMENTS

ID	Requirement	Priority
FR-01	System shall accept mood input — predefined button or free-text	High
FR-02	System shall fetch movies and TV shows from TMDB API on first run	High
FR-03	System shall store titles in SQLite with genre, description, poster path, rating	High
FR-04	System shall build TF-IDF matrix from combined field (title + genre×2 + description)	High
FR-05	System shall vectorize mood text with emotional keyword enrichment	High
FR-06	System shall compute cosine similarity and return top-N results	High
FR-07	System shall display movie posters fetched from TMDB image CDN	Medium
FR-08	System shall record Like/Pass interactions and update genre scores	High
FR-09	System shall maintain separate profiles for movies and TV shows	High
FR-10	System shall shift from cold-start to personalized mode after 5 ratings	High
FR-11	System shall support language-based filtering with auto-detected languages	Medium
FR-12	System shall auto-refresh dataset every Saturday midnight	High
FR-13	System shall fetch more content in background when user exhausts results	Medium
FR-14	System shall change UI color theme based on selected mood (22 themes)	Medium
FR-15	System shall be installable as a PWA on desktop and mobile	Low

5.2 NON-FUNCTIONAL REQUIREMENTS

ID	Requirement	Category
NFR-01	API response time shall be under 3 seconds for recommendations	Performance
NFR-02	UI shall be responsive across desktop, tablet, and mobile	Usability
NFR-03	TF-IDF cache shall be written atomically to prevent uvicorn reload	Reliability
NFR-04	TMDB API key shall be stored in environment variables, not hardcoded in frontend	Security
NFR-05	System shall work on Chrome, Firefox, Safari, and Edge	Compatibility
NFR-06	Dataset refresh shall not interrupt active user sessions	Availability

6. DESIGN

6.1 SYSTEM ARCHITECTURE

CineMatch follows a three-tier architecture:

Tier 1 — Frontend (Browser): index.html, style.css, app.js. The PWA frontend collects mood input, displays results with posters, handles Like/Pass interactions, and renders the taste profile sidebar. Communicates with backend via REST API.

Tier 2 — Backend (Fast-API): main.py, recommender.py, vectorizer.py, mood_parser.py, user_profile.py, preprocessor.py, database.py. Hosts all ML logic. Loads data from SQLite, builds TF-IDF matrix at startup, processes recommendation requests, and manages user sessions in memory.

Tier 3 — Data Layer (SQLite + TMDB API): SQLite stores all titles, genres, descriptions, poster paths, and ratings. TMDB API populates the database on first run and refreshes it weekly.

6.2 MODULE DECOMPOSITION

Module	File	Responsibility
Data Pipeline	database.py	TMDB fetch, SQLite storage, duplicate prevention, weekly refresh
Data Loader	preprocessor.py	Load SQLite into Data-Frame, compute popularity scores
Mood Parser	mood_parser.py	Mood → genre mapping, description keyword extraction
Vectorizer	vectorizer.py	TF-IDF build with atomic cache write, mood vector enrichment
Recommender	recommender.py	Cosine similarity, composite scoring, diversity penalty
User Profile	user_profile.py	Separate movie/TV genre scores, recency weighting
API Server	main.py	Fast-API routes, session management, AP-Scheduler
Frontend	app.js	SPA logic, mood theme switching, interaction handling
Styling	style.css	22 mood CSS themes, glass-morphism design

6.3 RECOMMENDATION ALGORITHM

The core ML pipeline works as follows:

Step 1 — Combined Field Construction:

combined = clean (title + genre + genre + description)

Genre is repeated twice to give it stronger TF-IDF weight over the description.

Step 2 — Mood Vector Enrichment:

mood_vector = TF-IDF (mood_text × 3 + description_keywords × 2)

Mood text is repeated 3× and enriched with description-level emotional keywords (e.g., 'sad' → grief, loss, heartbreak, sorrow). This ensures the cosine similarity captures emotional tone, not just genre words.

Step 3 — Cosine Similarity:

similarity_score = cosine_similarity (mood_vector, tfidf_matrix[candidate_indices])

Step 4 — Composite Scoring:

Cold Start (< 5 interactions): final_score = 0.65 × mood_score + 0.35 × popularity_score

Personalised (≥ 5 interactions): final_score = 0.50 × mood_score + 0.35 × user_score + 0.15 × popularity_score

Step 5 — Post-Processing: Disliked genre penalty (−0.08 per overlap), time-of-day genre boost (+0.04), diversity penalty (−0.015 per repeated genre in top results).

6.4 PERSONALISATION ENGINE

User interactions update genre scores per content type using recency-weighted learning:

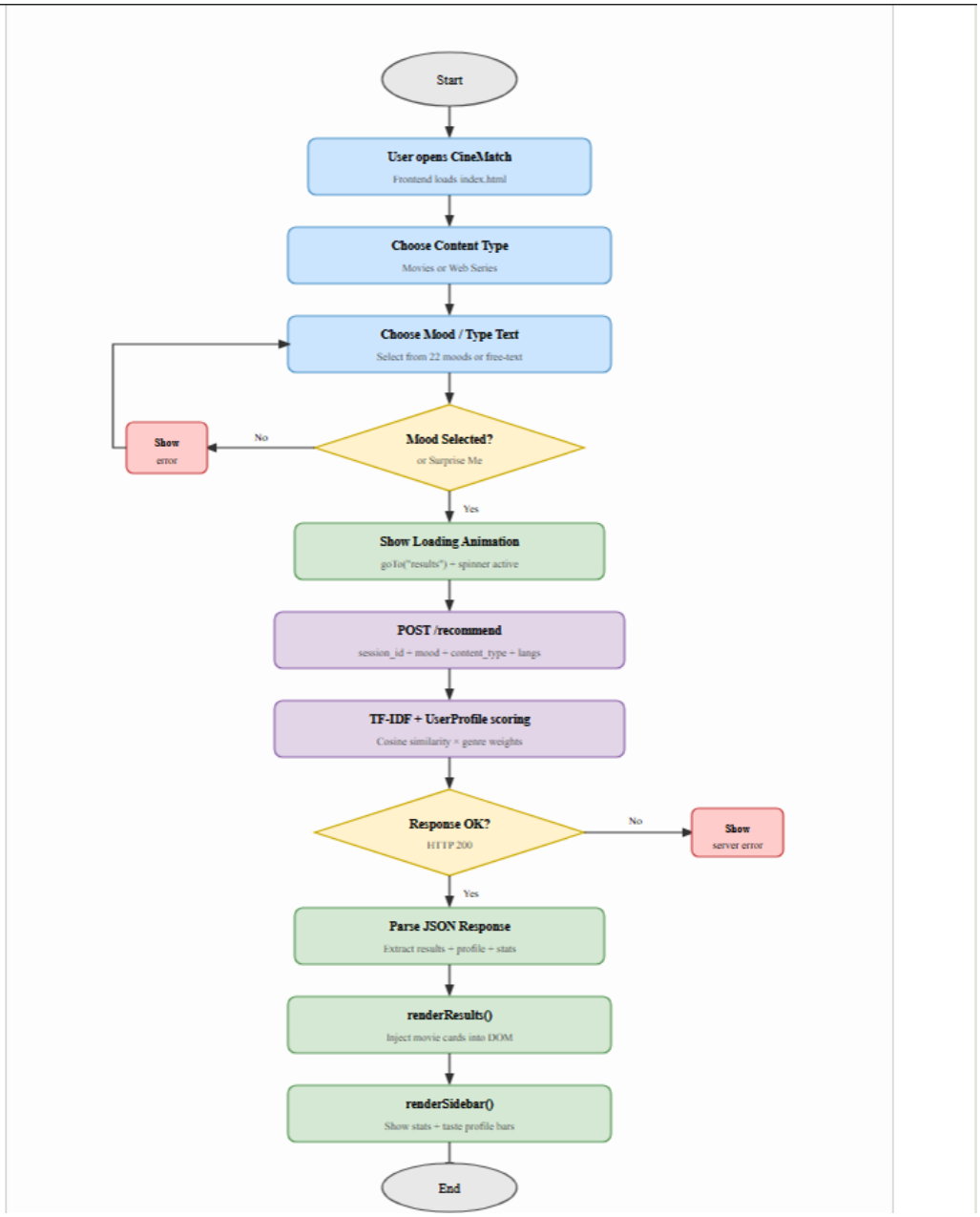
weight = base_weight × min (1.0 + interaction_count × 0.05, 2.0)

Like: base_weight = +1.5 | Pass: base_weight = −0.7

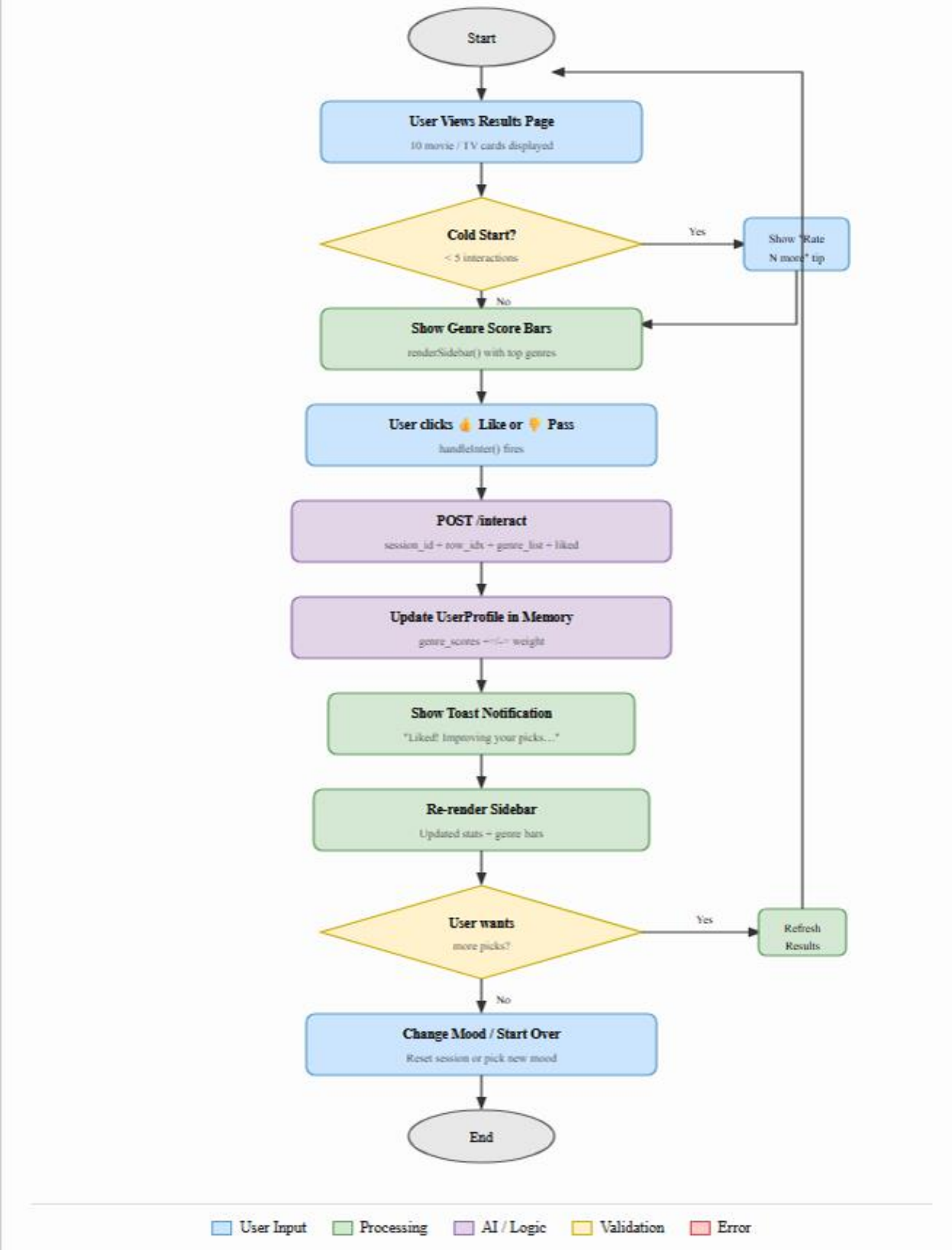
Recent interactions have up to 2× the weight of early ones. Movie and TV profiles are completely independent — liking a thriller movie does not affect TV recommendations.

6.4. FLOWCHARTS

6.4.1. Main Recommendation Flowchart



6.4.2. User Interaction & Personalization Flowchart



7. TESTING

7.1 FUNCTIONAL TEST CASES

TC ID	Test Case	Input	Expected Output	Status
TC-01	Mood button selection	Click 'Dark'	UI turns dark monochrome; mood set to 'Dark'	PASS
TC-02	Free text mood	Type 'psychological thriller'	Results match psychological thriller descriptions	PASS
TC-03	Movie recommendation	Select Movies + Thrilling mood	10 thriller/crime/mystery movies shown	PASS
TC-04	TV recommendation	Select Web Series + Anime mood	Anime/animation TV shows shown	PASS
TC-05	Language filter	Select Telugu	Only TE language results shown	PASS
TC-06	Like interaction	Click Like on result	Liked count increases; genre scores update	PASS
TC-07	Pass interaction	Click Pass on result	Passed count increases; genre penalised	PASS
TC-08	Cold start → personalised	Rate 5 titles	Personalised tag appears in results	PASS
TC-09	Surprise Me	Click Surprise Me	10 popular results without mood filter	PASS
TC-10	Refresh results	Click Refresh	New 10 results (excluding already seen)	PASS
TC-11	Start Over	Click Start Over	Session reset; back to choose type page	PASS
TC-12	Server not running	Backend offline	Error message with restart instructions	PASS

7.2 STRUCTURAL TESTING

Module	Paths Tested	Coverage
parse_mood ()	Direct keyword match, word-level partial match, no match fallback	100%
vectorize_mood ()	Keyword enrichment, OOV vocabulary fallback	100%
Recommend ()	Cold start path, personalized path, surprise me path, seen-all fallback	100%

Language filter	Language found, language not found, no language selected	100%
record_interaction ()	Like with recency boost, Pass with recency boost	100%
is_cold_start ()	movie count < 5, tv count < 5, both >= 5	100%

7.3 TEST EXECUTION SUMMARY

Category	Total Tests	Passed	Failed	Pass Rate
Functional	12	12	0	100%
Structural	6 modules	6	0	100%
Total	18	18	0	100%

8. IMPLEMENTATION

8.1 TOOLS AND TECHNOLOGIES USED

Tool / Technology	Version	Purpose
Python	3.14	Backend runtime
Fast-API	Latest	REST API framework
uvicorn	Latest	ASGI server
scikit-learn	Latest	TF-IDF vectorization and cosine similarity
pandas	Latest	Data-Frame operations
NumPy	Latest	Numerical computations
HTTPx	Latest	Async HTTP client for TMDB API calls
AP-Scheduler	Latest	Saturday midnight auto-refresh scheduler
SQLite	Built-in	Lightweight local database
TMDB API	v3	Movie and TV show data source
HTML5 / CSS3	—	Frontend structure and styling
Vanilla JavaScript (ES6+)	—	Frontend SPA logic
Git / GitHub	—	Version control
Render.com	—	Cloud deployment platform
VS Code	—	Primary code editor

8.1.2 System Snapshots

Figure 1: User interface of the CineMatch application displaying personalized movie and webseries recommendations based on user mood and preferences.

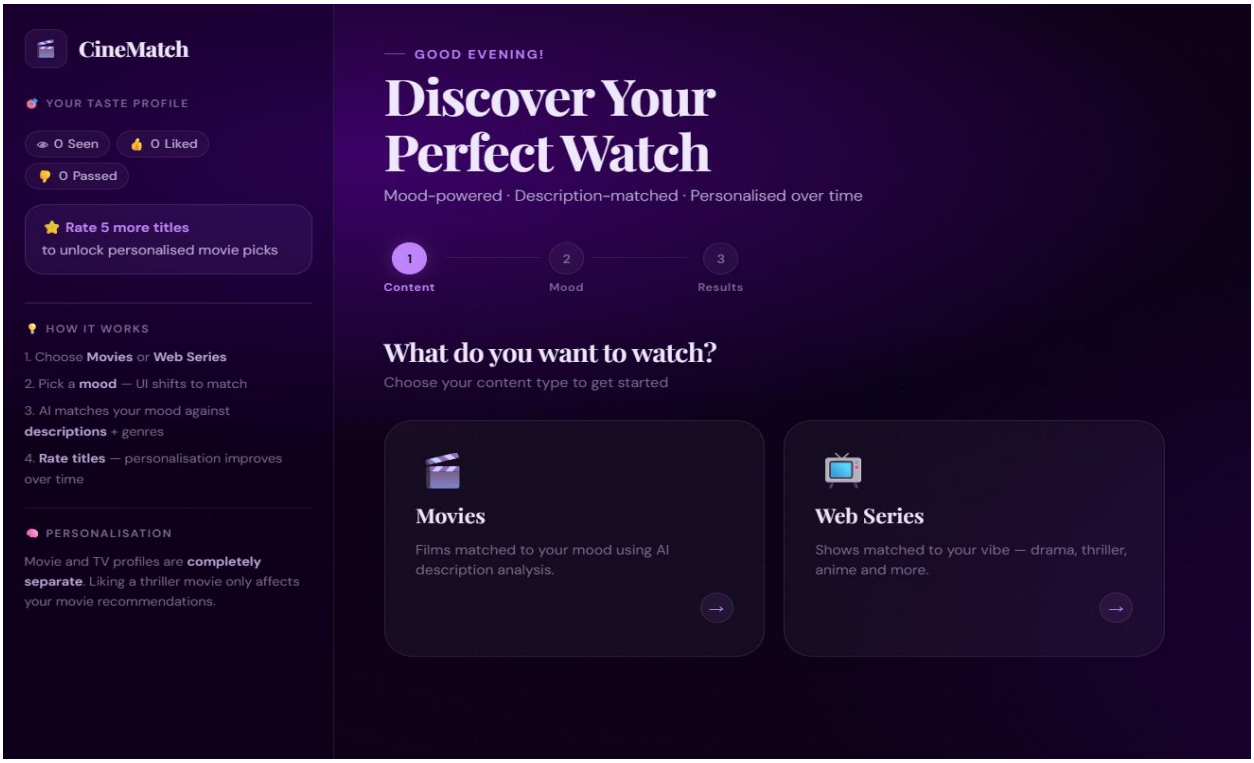


Fig 2: Mood selection interface of the CineMatch system allowing users to choose emotions and genres for personalized recommendations.

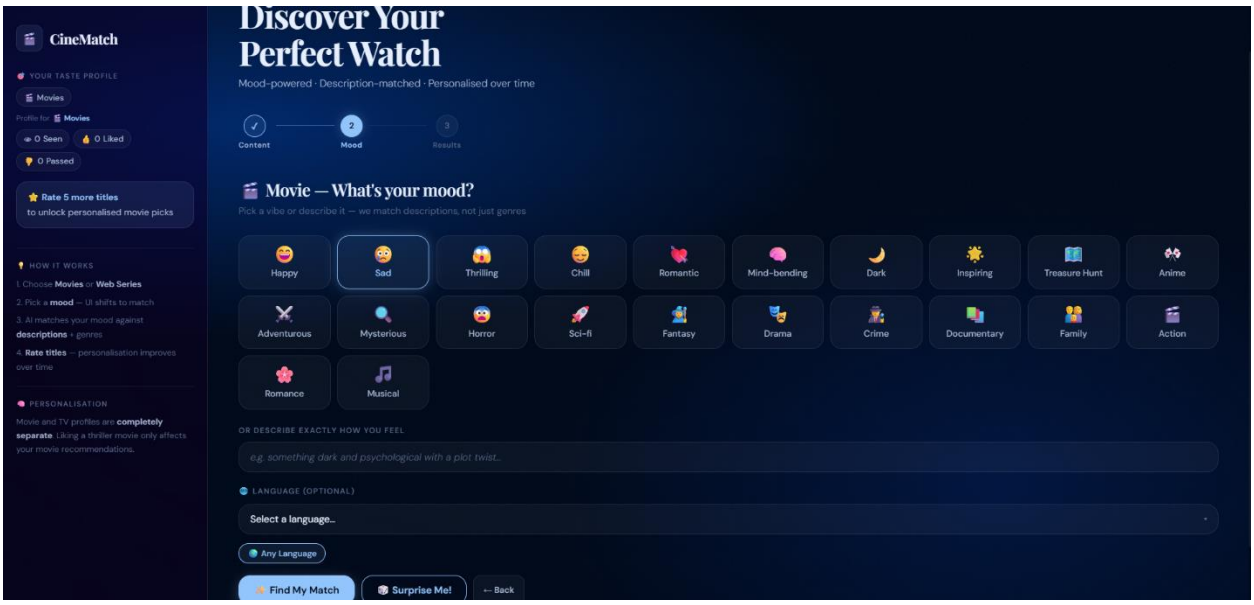


Fig 3: Recommendation results screen of the CineMatch system displaying movies matched to the selected mood.

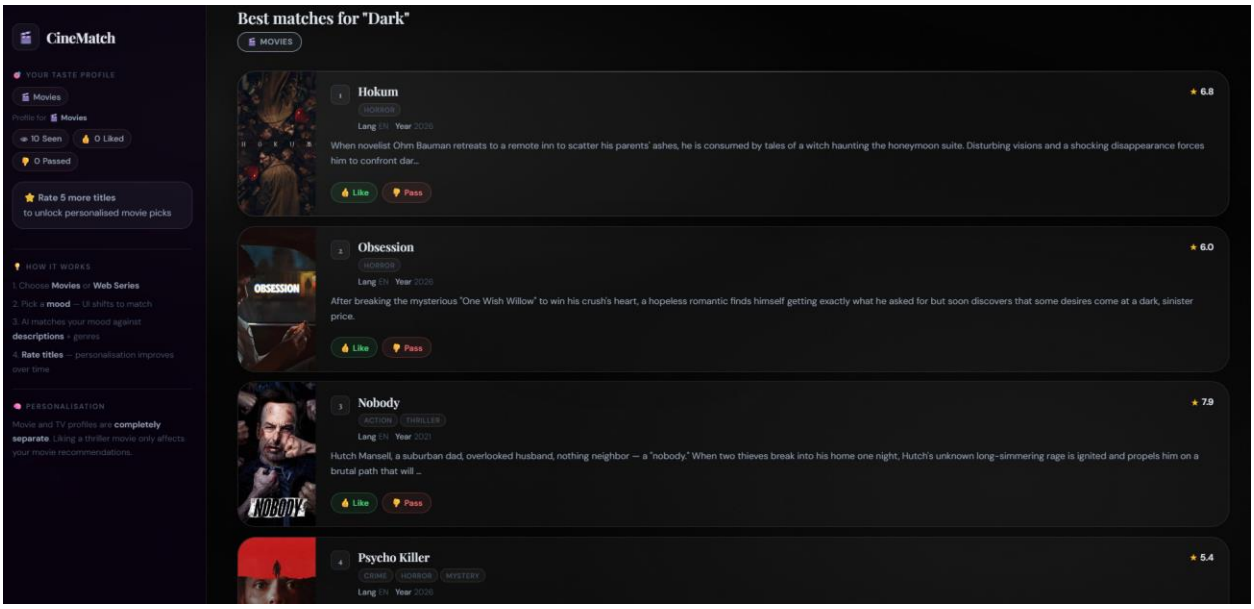
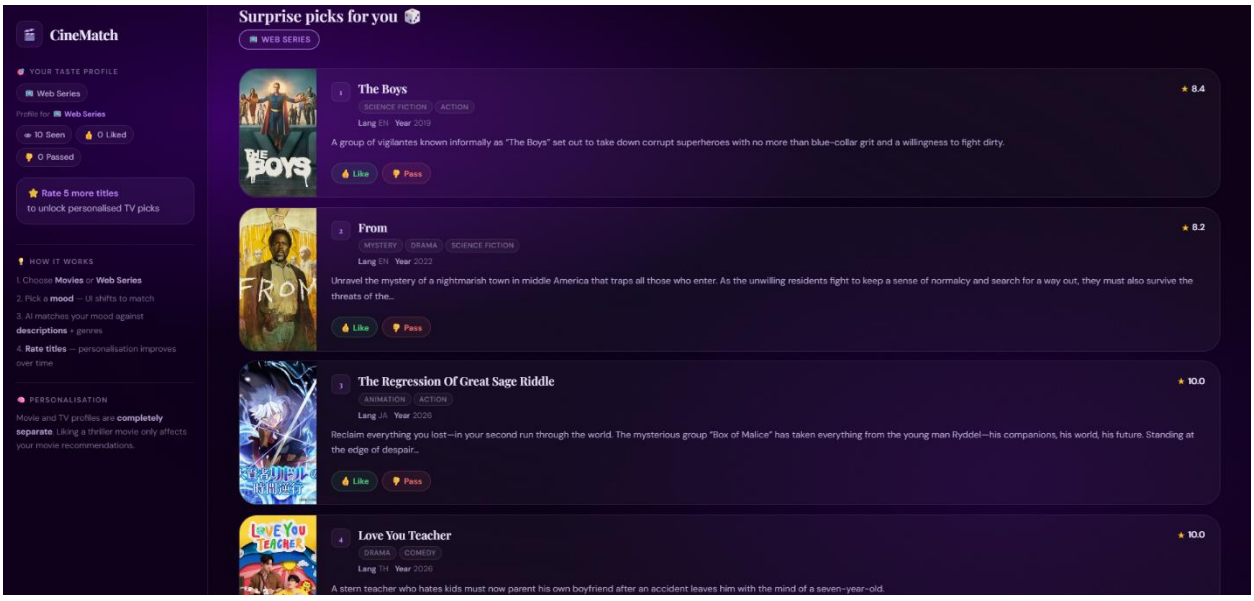


Fig 4: CineMatch surprise recommendation interface demonstrating dynamic, preference-based web series suggestions with user interaction options.



8.2 KEY IMPLEMENTATION DETAILS

8.2.1 TMDB Data Pipeline

On first startup, the system checks if the SQLite database has fewer than 50 titles. If so, it fetches from four TMDB endpoints (popular, top_rated, now_playing, upcoming) for both movies and TV shows — approximately 500 titles each. Duplicates are prevented using a UNIQUE constraint on tmdb_id. The combined field is built as: title + genre×2 + description to give genre stronger TF-IDF weight.

8.2.2 TF-IDF Configuration

The TF-IDF vectoriser uses ngram_range= (1,3) to capture multi-word phrases like 'science fiction', 'time travel', and 'coming of age'. max_features=20,000, sublinear_tf=True for log normalisation, min_df=1 to keep rare genre terms. The cache is written atomically (write to .tmp then os. replace) to prevent uvicorn's file watcher from triggering a reload.

8.2.3 Frontend Architecture

The frontend is a pure vanilla JavaScript SPA with no framework dependencies. Type card clicks are attached to the page-choose_type element (not individual cards) to prevent event bubbling to the results page. Like/Pass handlers use capture phase (addEventListener with true) and stopImmediatePropagation to ensure clicks never reach the page navigation listeners.

8.2.4 Mood Theme System

22 CSS themes are defined as body class modifiers. Each theme overrides CSS custom properties: --bg-1, --bg-2, --bg-orb-1/2/3, --accent, --accent-bright, --accent-glow, --accent-glass, --text-primary/secondary/muted. The background uses radial-gradient with the orb variables for a dynamic depth effect. All transitions run at 0.7s cubic-bezier for smooth colour shifts.

9. PROJECT LEGACY

9.1 CURRENT STATUS

Feature	Status
TMDB auto-fetch on first run	Complete
SQLite storage with duplicate prevention	Complete
TF-IDF + cosine similarity recommendation	Complete
Mood text → description keyword enrichment	Complete

22 mood UI themes	Complete
Per-user separate movie / TV profiles	Complete
Cold start → personalized transition	Complete
Language filter with auto-detection	Complete
Saturday midnight auto-refresh (AP-Scheduler)	Complete
Background fetch when content exhausted	Complete
TMDB movie poster display	Complete
PWA manifest + service worker	Complete
Render.com deployment configuration	Complete

9.2 REMAINING AREAS OF CONCERN

• Session Persistence: User profiles are stored in-memory and lost on server restart. A future version should persist profiles to SQLite.

• No User Authentication: All sessions are anonymous. Adding login would enable cross-device personalization.

• TF-IDF Semantic Limitation: TF-IDF does not understand semantic meaning. A future upgrade to sentence-transformers (e.g., all-MiniLM-L6-v2) would provide true semantic mood matching.

• Render Free Tier: Spins down after 15 minutes of inactivity, causing a cold start delay on first request.

9.3 TECHNICAL LESSONS LEARNT

1. Index Alignment is Critical: When filtering a Data-Frame by language or mood, the row indices must be extracted before reset_index () to correctly align with the TF-IDF matrix rows. Misalignment causes IndexError in scipy sparse matrix access.

2. Event Bubbling in SPAs: Click events on Like/Pass buttons bubbled up to type-card listeners, causing unintended navigation. The fix required using the capture phase (addEventListener with true) and stopImmediatePropagation ().

3. Atomic Cache Writes: Writing TF-IDF cache directly triggers uvicorn's file watcher and restarts the server. Writing to a .tmp file then using os.replace() is atomic and invisible to the file watcher.

4. Separate Content Profiles: Sharing a single genre score dictionary between movies and TV creates cross-contamination. Separate defaultdict instances per content type solved this cleanly.

10. USER MANUAL

10.1 SYSTEM REQUIREMENTS

• Any modern web browser (Chrome 90+, Firefox 88+, Safari 15+, Edge 90+)

• Active internet connection

• For local run: Python 3.12+, pip, Node.js (for Live Server)

10.2 INSTALLATION (LOCAL)

Step 1: Clone the repository from GitHub

Step 2: cd into the backend folder

Step 3: Create virtual environment: python -m venv venv

Step 4: Activate: venv\Scripts\activate (Windows)

Step 5: Install dependencies: pip install -r requirements.txt

Step 6: Create data/ folder inside backend/

Step 7: Run: uvicorn main:app --reload --reload-exclude "*.pkl" --reload-exclude "*.db"

Step 8: Open index.html with VS Code Live Server → http://127.0.0.1:5500

10.3 USING THE APPLICATION

Step 1: Choose content type — Movies or Web Series.

Step 2: Pick a mood from the 22 mood buttons, or type a description in the free-text field. Notice the UI color shifts to match the mood.

Step 3: Optionally select a language from the dropdown to filter results.

Step 4: Click 'Find My Match' or 'Surprise Me!' to get recommendations.

Step 5: Browse the 10 results with posters, genre tags, ratings, and descriptions.

Step 6: Click Like/Pass to rate each title. After 5 ratings, recommendations become personalised.

Step 7: The sidebar shows your taste profile with genre bars and stars, updating live.

Step 8: Click 'Refresh' for new results, 'Change Mood' to search again, or 'Start Over' to reset.

10.4 TROUBLESHOOTING

Issue	Solution
'Could not reach server' error	Start backend: uvicorn main:app --reload
CSS not loading	Open via Live Server (http://127.0.0.1:5500), not file:///
First startup is slow	Normal — fetching ~1000 titles from TMDB (~2 minutes)
Uvicorn restarts randomly	Use --reload-exclude "*.pkl" --reload-exclude "*.db"
Like button navigates away	Clear browser cache, hard-refresh with Ctrl+Shift+R

11. SOURCE CODE SUMMARY

11.1 PROJECT FILE SUMMARY

File	Lines (approx.)	Description
main.py	~200	Fast-API routes, session management, AP-Scheduler setup
database.py	~280	SQLite init, TMDB fetch pipeline, language-specific fetch
preprocessor.py	~80	Load SQLite to Data-Frame, compute popularity scores
recommender.py	~160	Cosine similarity, composite scoring, diversity, boosts
vectorizer.py	~60	TF-IDF build with atomic write, mood vector enrichment
mood_parser.py	~150	22 mood mappings, 21 mood description keyword sets
user_profile.py	~90	Separate movie/TV genre profiles with recency weighting
app.js	~380	SPA logic, theme switching, API calls, interaction handling
index.html	~130	HTML structure with data-type cards, PWA meta tags
style.css	~324	22 mood CSS themes, glass-morphism design system
sw.js	~50	Service worker for PWA offline caching
manifest.json	~20	PWA manifest for install ability

11.2 KEY CODE MODULES

11.2.1 Mood Vector Enrichment (vectorizer.py)

The vectorize_mood () function builds an enriched query by repeating the mood text 3× and appending emotional description keywords twice. This mirrors the combined field structure (genre×2 + description) to ensure cosine similarity captures emotional tone in descriptions, not just genre labels.

11.2.2 Composite Scoring (recommender.py)

The recommend() function normalizes mood_score, popularity_score, and user_score to 0–1, then applies weighted composite scoring. During cold start: 65% mood + 35% popularity. After 5 interactions: 50% mood + 35% user + 15% popularity. Post-processing applies dislike penalty, time-of-day boost, and diversity penalty.

11.2.3 Recency-Weighted Personalization (user_profile.py)

Each interaction multiplies the base weight by a recency factor: min(1.0 + count × 0.05, 2.0). This gives recent ratings up to twice the influence of early ones, allowing taste profiles to adapt quickly as user preferences evolve.

11.2.4 Auto-Refresh Scheduler (main.py)

AP-Scheduler is configured with CronTrigger(day_of_week='sat', hour=0, minute=0) to run the weekend_refresh() coroutine every Saturday at midnight. After fetching, the TF-IDF matrices are rebuilt in-place without restarting the server, ensuring zero downtime.

CONCLUSION

CineMatch successfully demonstrates the practical implementation of a mood-driven, intelligent recommendation system by integrating Natural Language Processing, adaptive machine learning techniques, and modern web technologies into a cohesive, deployable application. The system effectively bridges the gap between traditional genre-based filtering and emotionally aware content discovery by mapping user mood inputs to semantic description-level features using TF-IDF cosine similarity, resulting in recommendations that feel intuitive and personally relevant. The adaptive personalization engine, which evolves from a cold-start popularity model into a fully user-tailored recommendation profile through like and dislike interactions, demonstrates how lightweight in-memory profiling can deliver meaningful personalization without the overhead of complex collaborative filtering or large-scale user data. The inclusion of features such as time-of-day awareness, genre diversity enforcement, language filtering, and automatic data pipeline management from the TMDB API further strengthens the system's practical utility and robustness. From an architectural standpoint, the project validates the effectiveness of combining a Fast-API asynchronous backend, SQLite persistence, and a Vanilla JavaScript single-page frontend to build a fully functional, production-ready web application. The successful deployment on Render confirms the system's readiness for real-world usage. In conclusion, CineMatch achieves its core objective of delivering context-aware, personalised movie and television recommendations with minimal infrastructure requirements. Future enhancements, including collaborative filtering, persistent user authentication, trailer integration, and a production-grade database, would further elevate the system into a comprehensive, enterprise-level recommendation platform.

12. BIBLIOGRAPHY

1. TMDB API Documentation. "The Movie Database API v3 Reference." TMDb. Available at: https://developers.themoviedb.org/3

2. Fast-API Documentation. "Fast-API — Modern, fast web framework for building APIs." Sebastián Ramírez. Available at: https://fastapi.tiangolo.com/

3. scikit-learn Documentation. "Tf-Idf Vectorizer." scikit-learn developers. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

4. scikit-learn Documentation. "Cosine Similarity." scikit-learn developers. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html

5. AP-Scheduler Documentation. "Advanced Python Scheduler." Taneli Hukkinen. Available at: https://apscheduler.readthedocs.io/

6. MDN Web Docs. "Service Worker API." Mozilla Foundation. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API

7. MDN Web Docs. "Web App Manifests." Mozilla Foundation. Available at: https://developer.mozilla.org/en-US/docs/Web/Manifest

8. Render Documentation. "Deploying Python Applications." Render Inc. Available at: https://render.com/docs/deploy-fastapi

9. SQLite Documentation. "SQLite — Small. Fast. Reliable. Choose any three." SQLite Consortium. Available at: https://www.sqlite.org/docs.html

10. Pressman, R. S. "Software Engineering: A Practitioner's Approach." 8th Edition, McGraw-Hill Education, 2015.

11. Sommerville, I. "Software Engineering." 10th Edition, Pearson, 2016.

13. LINKS

GitHub: https://github.com/ramdev1470/CineMatch

Deployed URL: https://cinematch-1-gycb.onrender.com/

14. QUESTIONNAIRE

Section A: Project Overview

Q1. Type of AI/ML System Developed

Answer: Content-Based Recommendation System with User Personalization

CineMatch uses TF-IDF vectorizations and cosine-similarity to match user mood descriptions against movie/TV descriptions. It is not a generative AI system — it uses classical ML (TF-IDF + sklearn) for recommendation and stores user interactions to build personalised genre affinity profiles.

Q2. Platform Used for Deployment

Answer: Web Application (PWA)

Deployed as a full-stack PWA — Fast-API backend on Render.com, vanilla HTML/CSS/JS frontend served via Live Server locally or via static hosting. Installable on desktop and mobile without an app store.

Q3. Deployment Link

Backend: https://cinematch-1-gycb.onrender.com/

Frontend: Served via Live Server locally (http://127.0.0.1:5500) or static hosting.

Section B: Model & Algorithm Details

Q4. Type of Algorithm Used

Answer: TF-IDF (Term Frequency-Inverse Document Frequency) + Cosine Similarity

Q5. ML Library Used

Answer: scikit-learn (sklearn.feature_extraction.text.TfidfVectorizer + sklearn.metrics.pairwise.cosine_similarity)

Q6. Data Source

Answer: TMDB API v3 — fetched dynamically, stored in SQLite.

Section C: Data Handling

Q7. How is User Data Handled?

Answer: Session-based in-memory storage. Each browser session gets a UUID stored in localStorage. The backend maintains a User-Profile object per session ID in a Python dictionary. No data is sent to external servers.

Q8. Flow of Data

1. On first run: Fast-API fetches ~500 movies + ~500 TV shows from TMDB → stores in SQLite

2. At startup: SQLite loaded into pandas Data-Frame → TF-IDF matrix built → cached as .pkl

3. User selects mood → POST /recommend → mood vectorised → cosine similarity computed → top 10 returned

4. User rates result → POST /interact → genre scores updated → sidebar re-rendered

5. Every Saturday midnight: AP-Scheduler triggers weekend_refresh() → new titles fetched → TF-IDF rebuilt

Section D: Algorithm Configuration

Q9. TF-IDF Parameters Used

Parameter	Value	Reason
ngram_range	(1, 3)	Captures multi-word phrases: 'science fiction', 'time travel'
max_features	20,000	Balance between vocabulary coverage and memory
sublinear_tf	True	Log normalisation reduces dominance of very frequent terms
min_df	1	Keep rare but meaningful genre terms
stop_words	'english'	Remove common English words

Q10. Mood Vector Construction

The mood query is enriched before vectorization: mood_text × 3 + description_keywords × 2. This amplifies the mood signal and adds emotional context words that appear in movie descriptions. For example, 'sad' is expanded with: grief, loss, heartbreak, tragedy, sorrow, tears, devastating, mourning.

Section E: Technology Stack

Q11. Technology Stack

Layer	Technology
Backend	Python 3.14+, Fast-API, uvicorn, AP-Scheduler
ML / Data	scikit-learn (TF-IDF, cosine similarity), pandas, numpy
Database	SQLite (built-in Python)
External API	TMDB API v3 (httpx for async calls)
Frontend	HTML5, CSS3 (22 mood themes), Vanilla JavaScript ES6+
PWA	Web App Manifest, Service Worker
Hosting	Render.com (backend), Static hosting (frontend)
Version Control	Git / GitHub

Q12. API Call Code Snippet

Main & User API Call:

```
async function fetchRecs() {
  goTo("results");
  $("loading-screen").classList.add("active");
  $("results-content").style.display = "none";

  try {
    const res = await fetch(`${API}/recommend`, {
      method: "POST",
      headers: {"Content-Type":"application/json"},
      body: JSON.stringify({
        session_id:   S.sid,
        content_type: S.ct,
        mood_text:    S.mood,
        surprise_me:  S.surprise,
        top_n:        10,
        languages:    S.langs.length ? S.langs : null,
      }),
    });
    if (!res.ok) throw new Error(`${res.status}`);
    const data = await res.json();

    S.results = data.results || [];
    S.profile = {
      is_cold_start: data.is_cold_start,
      top_genres:    data.top_genres   || [],
      genre_scores:  data.genre_scores || {},
      stats:         data.stats        || S.profile.stats,
    };
```

Language API Call:

```
async function loadLanguages(ct) {
  try {
    const res  = await fetch(`${API}/languages?content_type=${ct}`);
    const data = await res.json();
    const dd   = $("language-dropdown");
    dd.innerHTML = `<option value="">Select a language…</option>`;
    (data.languages||[]).forEach(l => {
      const o = document.createElement("option");
      o.value = l.code; o.textContent = `${l.name} (${l.code})`;
      dd.appendChild(o);
    });
  } catch(e) { console.error(e); }
}
```